Documento de Arquitectura de Software (DAS / SAD)

arc42 simplificado + Modelo C4 + ADRs + Corte Vertical

SITRA

Servicio de Integración y Trazabilidad

Exposición segura y auditable de un núcleo transaccional legado a una billetera digital externa

Caso base: Caso 5 — Cooperativa de ahorro y crédito

Curso: Arquitectura de Software — Proyecto individual
Estudiante: Jorge Mario Vélez Vásquez
Proyecto: SITRA — Servicio de Integración y Trazabilidad
Repositorio: https://github.com/JorgeMario-V/sitra
Versión del documento: 0.4 — Fecha: 17 / 10 / 2026

Historial de versiones

Versión	Fecha	Contenido	Clase
0.1	26/09/2026	Ficha de drivers revisada + Sección 1 (visión, historias de usuario, atributos de calidad, restricciones).	Clase 1
0.4	17/10/2026	Sección 2 (patrón N-Capas y pila tecnológica); Sección 3.1 y 3.2 (contexto y contenedores); Sección 3.3 y 3.4 (clases y secuencia); Sección 5.2 (script SQL DDL). Ajuste en 1.3: nombre y fórmula de la métrica principal.	Clases 2, 3 y 4
0.5	24/10/2026	Sección 4 (ADR), Sección 3.5 (despliegue) y arranque del corte vertical.	Clase 5
1.0	21/11/2026	Documento completo (Secciones 1 a 6) + repositorio.	Clase 8

SECCIÓN 1: INTRODUCCIÓN Y DRIVERS ARQUITECTÓNICOS

1.1 Visión general del negocio y alcance

Problema de negocio

Una cooperativa de ahorro y crédito opera sobre un núcleo transaccional construido hace 20 años en un lenguaje propietario, cuya documentación se perdió y que nadie del equipo actual escribió. El regulador le exige dos cosas en un plazo fijo de 6 meses: integrarse con una billetera digital externa y entregar trazabilidad de sus transacciones. Si no cumple, arriesga la licencia de operación.

Hoy el problema lo padecen el área de cumplimiento y la gerencia. Sin software, la evidencia para el auditor se reconstruye a mano con reportes parciales del núcleo y conciliaciones en hojas de cálculo: un proceso lento, que no demuestra que los datos no fueron alterados y que no alcanza los tiempos del auditor. La exposición de funciones a un tercero, simplemente, no existe.

Además, el cliente fijó tres condiciones que no se discuten: el núcleo no se va a cambiar (lo seguirán usando porque su negocio no es el software), no hay dinero adicional ni se invertirá en APIs (solo se trabaja con lo que se tiene) y el personal de la cooperativa se dedica a sus funciones, no a desarrollar ni mantener software.

Propuesta de solución

El sistema permite a la billetera digital externa, autenticada, consultar saldos y registrar transferencias sobre cuentas autorizadas de la cooperativa, para entregar al auditor el 100 % de esas operaciones con evidencia verificable de que no fueron alteradas, sin modificar el núcleo legado y sin inversión adicional.

Alcance (in-scope)

- Conector externo al núcleo que usa solo lo que el núcleo ya tiene hoy (sus salidas e interfaces actuales); no se compra ni se instala ningún producto.
- Exposición de dos operaciones a la billetera: consulta de saldo y registro de transferencias.
- Autenticación de la billetera y autorización por rol (BILLETERA y AUDITOR).
- Lista blanca de cuentas autorizadas: solo esas cuentas son visibles para el tercero.
- Bitácora de auditoría que no se puede modificar ni borrar, consultable por el auditor.
- Simulador del núcleo para poder probar sin tocar el sistema en producción.

Fuera de alcance (out-of-scope)

- Modificar, reescribir o reemplazar el núcleo legado. Tampoco se plantea una migración gradual: el núcleo se queda indefinidamente.
- Cualquier otra función del núcleo distinta de saldo y transferencias.
- Aplicación móvil o interfaz para el usuario final: el consumidor es la billetera (sistema a sistema).
- Productos o licencias de pago: API Gateway comercial, servicios de nube facturados, herramientas de auditoría de pago.
- Alta disponibilidad (réplicas, balanceo) y reportería avanzada o tableros analíticos.
- Contratar soporte externo: no es una decisión de arquitectura; se eleva a la junta directiva (ver 1.4).

1.2 Historias de usuario arquitectónicamente significativas

ID	Historia de usuario	¿Por qué es arquitectónicamente significativa?
HU-01	Como billetera digital externa autenticada, quiero registrar una transferencia sobre una cuenta autorizada, para que se ejecute en el núcleo y quede con evidencia auditable.	Es la más riesgosa: recorre todas las capas y ejercita a la vez autenticación, validación, integración con el núcleo legado, persistencia y bitácora. Es el Corte Vertical (Sección 5).

ID	Historia de usuario	¿Por qué es arquitectónicamente significativa?
HU-02	Como billetera digital externa autenticada, quiero consultar el saldo de una cuenta autorizada, para mostrárselo al usuario final.	Ejercita la integración de solo lectura con el núcleo y el mínimo privilegio (lista blanca). Valida que el saldo no se copia: siempre se lee del núcleo, que es la única fuente de verdad.
HU-03	Como auditor, quiero consultar las operaciones por cuenta y rango de fechas y verificar su integridad, para entregar la evidencia que exige el regulador.	Materializa el atributo crítico del caso (trazabilidad). Ejercita el control por rol y el mecanismo que demuestra que un registro no fue alterado.

Criterio: las tres validan el patrón, la seguridad, la persistencia y la integración con un sistema externo. Se descartaron historias como "ver el estado del servicio" o "listar cuentas" por no ser significativas.

1.3 Atributos de calidad prioritarios (ISO/IEC 25010) — especificación cuantitativa

La prioridad la da el caso: seguridad y trazabilidad "por encima de cualquier otro atributo". La fiabilidad sigue porque se mueve dinero de asociados, y la mantenibilidad sube a primer orden porque no habrá personal técnico que sostenga el sistema.

#	Atributo (ISO 25010)	Escenario de calidad (estímulo → entorno → respuesta)	Métrica objetivo (medible)	¿Cómo se verificará?
AC-01	Seguridad — responsabilidad, no repudio, integridad (ficha: AC-06)	La billetera hace operaciones en operación normal; además, un cliente sin token intenta registrar una transferencia y alguien intenta modificar un registro de la bitácora directamente en la base de datos.	Métrica principal — Completitud del registro de auditoría (ISO/IEC 25023): X = A / B, donde A = operaciones registradas en la bitácora y B = operaciones realizadas. Meta: X = 1 (100 %). Además: 0 registros de la bitácora modificables o borrables; 401 en el 100 % de los intentos sin token; la consulta del auditor por cuenta y fechas responde en ≤ 5 s.	Comparar operaciones hechas contra registros de la bitácora (Postman); script db/pruebas/verificar_restricciones.sql (el UPDATE y el DELETE son rechazados); tiempo de respuesta medido en Postman.
AC-02	Fiabilidad — tolerancia a fallos (ficha: AC-05)	El núcleo legado no responde mientras se procesa una transferencia ya aceptada. El sistema la cierra sin dejarla a medias.	100 % de estas transferencias terminan en estado FALLIDA con respuesta 503, sin efecto parcial. Repetir la misma referencia responde 409 y nunca duplica la operación. Espera máxima al núcleo: 5 s.	Prueba de integración con el simulador del núcleo en modo "sin respuesta" + revisión en BD del estado y de que no hay duplicados.
AC-03	Mantenibilidad — modularidad y capacidad de ser probado (ficha: AC-07)	Se necesita probar sin el núcleo real (no hay ambiente de pruebas) o cambiar la forma de conectarse a él, y no hay personal técnico que conozca el núcleo.	El cambio toca como máximo 1 clase del conector y 1 valor de configuración, y 0 archivos de las capas de presentación, negocio y datos. Todas las pruebas corren sin conexión al núcleo real.	Revisión de código del corte vertical + ejecución de las pruebas con el perfil de pruebas (simulador) activo.

Nota: "trazabilidad" no es una característica propia de la ISO/IEC 25010. Se ubica dentro de Seguridad, en las sub-características responsabilidad (accountability) y no repudio. La métrica "completitud del registro de auditoría" está definida para esa sub-característica en la ISO/IEC 25023, la norma que dice cómo medir la 25010.

1.4 Restricciones

Ya vienen dadas por el cliente y el regulador; la arquitectura las acata.

Tipo	Restricción	Origen / justificación
Técnica	El núcleo legado no se modifica, ni ahora ni a futuro. Solo se consumen los puntos de acceso que ya expone.	Caso: 20 años, lenguaje propietario, sin documentación, alto riesgo de romper algo al tocarlo. Cliente: no se va a cambiar el sistema.
Técnica	No existe ambiente de pruebas del núcleo; toda prueba previa se hace contra un sustituto controlado.	Consecuencia de la anterior y de la falta de presupuesto: no hay con qué montar una copia del núcleo.
Técnica	Base de datos relacional de licencia libre y con transacciones; la exposición al tercero es una API REST con JSON sobre HTTPS.	Lineamiento del curso + la bitácora exige integridad; interoperabilidad con la billetera.
Operativa	Cero presupuesto: solo software libre e infraestructura que la cooperativa ya tiene. Nada de licencias, nube de pago ni productos de API.	Gerencia: "solo tenemos lo que se tiene"; no se invertirá en APIs.
Operativa	Cero personal técnico: la solución debe operarse por configuración y arrancar sola, sin nadie que la desarrolle o mantenga.	Cliente: el personal solo se dedica a las funciones de la cooperativa; nadie del equipo original.
De tiempo	Plazo regulatorio fijo de 6 meses. En lo académico: corte vertical funcionando el 07/11/2026 y documento final el 21/11/2026.	Regulador (licencia de operación) + cronograma del curso.
Normativa	Trazabilidad auditable y registros no alterables; datos personales tratados según la Ley 1581 de 2012 (habeas data).	Exigencia del auditor + marco colombiano.

Tensión declarada: el regulador obliga a conectar el núcleo con la billetera, pero el cliente no da dinero ni personal. Con literalmente nada nuevo no se puede cumplir. La arquitectura propone el mínimo indispensable —usar primero lo que el núcleo ya tiene y, donde no alcance, un conector pequeño con software gratuito sobre los equipos existentes— y deja en manos de la junta directiva quién lo opera y sostiene. Es la pregunta al cliente de la ficha de drivers.

SECCIÓN 2: ESTRATEGIA ARQUITECTÓNICA Y TECNOLÓGICA

2.1 Patrón arquitectónico principal: N-Capas

Se selecciona el patrón N-Capas (presentación, negocio y datos, más un conector del núcleo) porque el atributo de calidad prioritario AC-03 (Mantenibilidad) exige separar responsabilidades y poder cambiar la conexión con el núcleo por un simulador sin tocar la lógica de negocio, y porque la restricción operativa de cero personal técnico exige una estructura que cualquier desarrollador Java reconozca sin capacitación. Se descarta Hexagonal porque su costo en número de clases, interfaces y curva de aprendizaje contradice esa misma restricción de cero personal técnico; de él solo se toma una idea: el núcleo se usa a través de la interfaz NucleoConector. Se descarta Microservicios porque su costo en complejidad operativa contradice las restricciones de cero presupuesto y cero personal.

Patrón candidato	A favor en este caso	En contra en este caso	Veredicto
MVC monolítico	Muy simple y rápido de construir dentro de los 6 meses.	El controlador terminaría hablando directo con el núcleo y con la base de datos: no hay dónde poner el simulador sin tocar código de negocio.	Descartado: incumple la métrica de AC-03.
N-Capas	Cada capa tiene una sola responsabilidad. Es la estructura estándar de Spring Boot: fácil de leer para quien herede el sistema.	Algo más de código que MVC (servicios y repositorios separados).	SELECCIONADO, con un conector intercambiable para el núcleo.
Hexagonal	Máxima independencia del núcleo y de la base de datos; muy fácil de probar.	Más clases, interfaces y conceptos (puertos, adaptadores); difícil de sostener sin personal técnico.	Descartado como patrón completo; se toma solo la interfaz del conector.
Microservicios	Escala y despliega cada parte por separado.	Varios servicios que operar y monitorear; exige personal y presupuesto que no existen.	Descartado.

Capas y responsabilidades

Capa	Responsabilidad	Clases principales (paquete)
Seguridad	Rechazar peticiones sin token válido (401) y verificar el rol.	FiltroJwt, ConfiguracionSeguridad (security)
Presentación	Recibir la petición HTTP, validar el formato y devolver el código HTTP correcto. No tiene lógica de negocio.	TransferenciaController, SaldoController, AuditoriaController (controller)
Negocio	Reglas: cuenta autorizada, monto positivo, referencia no repetida, estados de la transferencia y registro en la bitácora.	TransferenciaService, AuditoriaService (service)
Datos	Guardar y leer de PostgreSQL.	TransferenciaRepository, RegistroAuditoriaRepository, … (repository) + entidades (model)
Conector del núcleo	Única parte que habla con el núcleo legado. Tiene dos versiones intercambiables por configuración: la real y el simulador.	NucleoConector (interfaz), NucleoLegadoConector, NucleoSimulador (integration)

Regla de dependencias: cada capa solo llama a la de abajo. El servicio no sabe si habla con el núcleo real o con el simulador; solo conoce la interfaz NucleoConector.

2.2 Pila tecnológica (Tech Stack)

Capa	Tecnología elegida	Versión	Alternativa descartada	Justificación (AC / restricción)
Lenguaje backend	Java (OpenJDK)	21 LTS	Node.js 20	Restricción de tiempo: dominio previo del estudiante en Java y Spring Boot. Licencia libre (cero presupuesto). Versión LTS: soporte largo, menos mantenimiento (AC-03).
Framework backend	Spring Boot	3.3.x	Express / Quarkus	AC-03: impone la estructura por capas (@RestController, @Service, @Repository) y permite perfiles de configuración (prod / test) para cambiar el conector por el simulador sin tocar código.
Seguridad	Spring Security + JWT + BCrypt	6.x	Sesiones en servidor	AC-01: el consumidor es otro sistema, sin navegador; el token identifica a quién llama y ese dato queda en la bitácora. Secretos guardados con BCrypt.
Acceso a datos	Spring Data JPA + Hibernate	3.3.x	JDBC / SQL a mano	AC-03: repositorios sin escribir SQL repetitivo; las entidades corresponden 1 a 1 con las tablas. Se acepta perder control fino del SQL.
Migraciones	Flyway	10.x	ddl-auto de Hibernate	AC-01: el esquema queda versionado (V1, V2) y se puede auditar cuándo cambió.
Base de datos	PostgreSQL	16	MongoDB	AC-01: llaves foráneas, CHECK, UNIQUE, transacciones y un trigger que impide modificar la bitácora. Licencia libre (cero presupuesto).
Huella de integridad	SHA-256 (incluido en Java)	JDK 21	Servicio de auditoría en la nube	Cero presupuesto: viene con el lenguaje, sin instalar ni pagar nada. Permite detectar si un registro fue alterado.
Conector del núcleo	Interfaz Java propia + simulador	—	API Gateway comercial	Cero presupuesto (sin licencias) + no hay ambiente de pruebas del núcleo: el simulador permite probar todo sin tocar producción.
Frontend	Ninguno en el corte vertical	—	SPA en React	Alcance: el consumidor es la billetera (sistema a sistema); el auditor usa la API. Postman demuestra el flujo completo.
Pruebas	JUnit 5 + Spring Boot Test + Postman	5.10	Solo pruebas manuales	Verificación de las métricas de AC-01, AC-02 y AC-03.
Despliegue	Docker + Docker Compose	26.x	Nube de pago	Cero presupuesto: corre en un servidor que la cooperativa ya tiene. Reinicio automático de contenedores (cero personal).
Diagramas	PlantUML	1.2024	Herramienta con licencia	Los diagramas son texto: se versionan en Git junto al código.
Control de versiones	Git + GitHub	—	—	Restricción del curso.

SECCIÓN 3: VISTAS DE ARQUITECTURA (MODELO C4 Y UML)

Los diagramas están en notación UML y se hicieron con PlantUML. Las fuentes (.puml) y las imágenes en alta resolución están en la carpeta docs/diagramas/ del repositorio.

3.1 Vista de Contexto — C4 Nivel 1

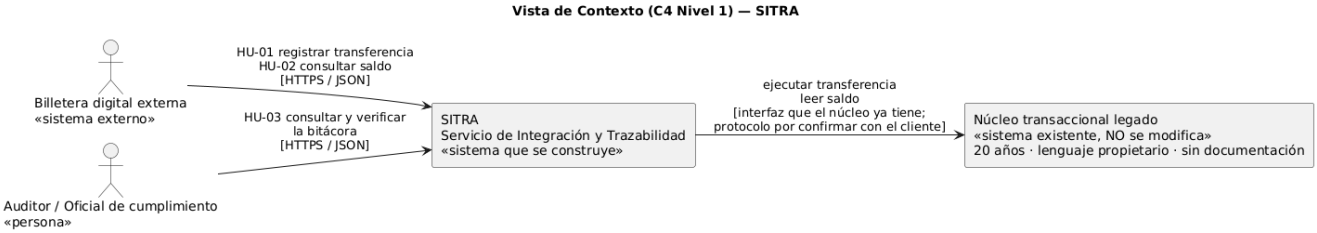


Figura 1. Vista de Contexto: SITRA como caja negra, sus actores y el sistema externo.

Leyenda

- Figura de persona: actor (persona o sistema que inicia una acción). Rectángulo: sistema.
- Texto en las flechas: historia de usuario que viaja por esa relación y, entre corchetes, el protocolo.

Lectura en 30 segundos

SITRA se ubica entre la billetera y el núcleo legado. La billetera solo puede consultar saldos y registrar transferencias (HU-01 y HU-02); el auditor solo consulta y verifica la bitácora (HU-03). Nadie de afuera habla directamente con el núcleo: solo SITRA, y usando la interfaz que el núcleo ya tiene. El protocolo exacto de esa interfaz se marca "por confirmar con el cliente" porque el caso no lo dice; averiguarlo es el primer paso de "trabajar con lo que se tiene".

3.2 Vista de Contenedores / Capas — C4 Nivel 2

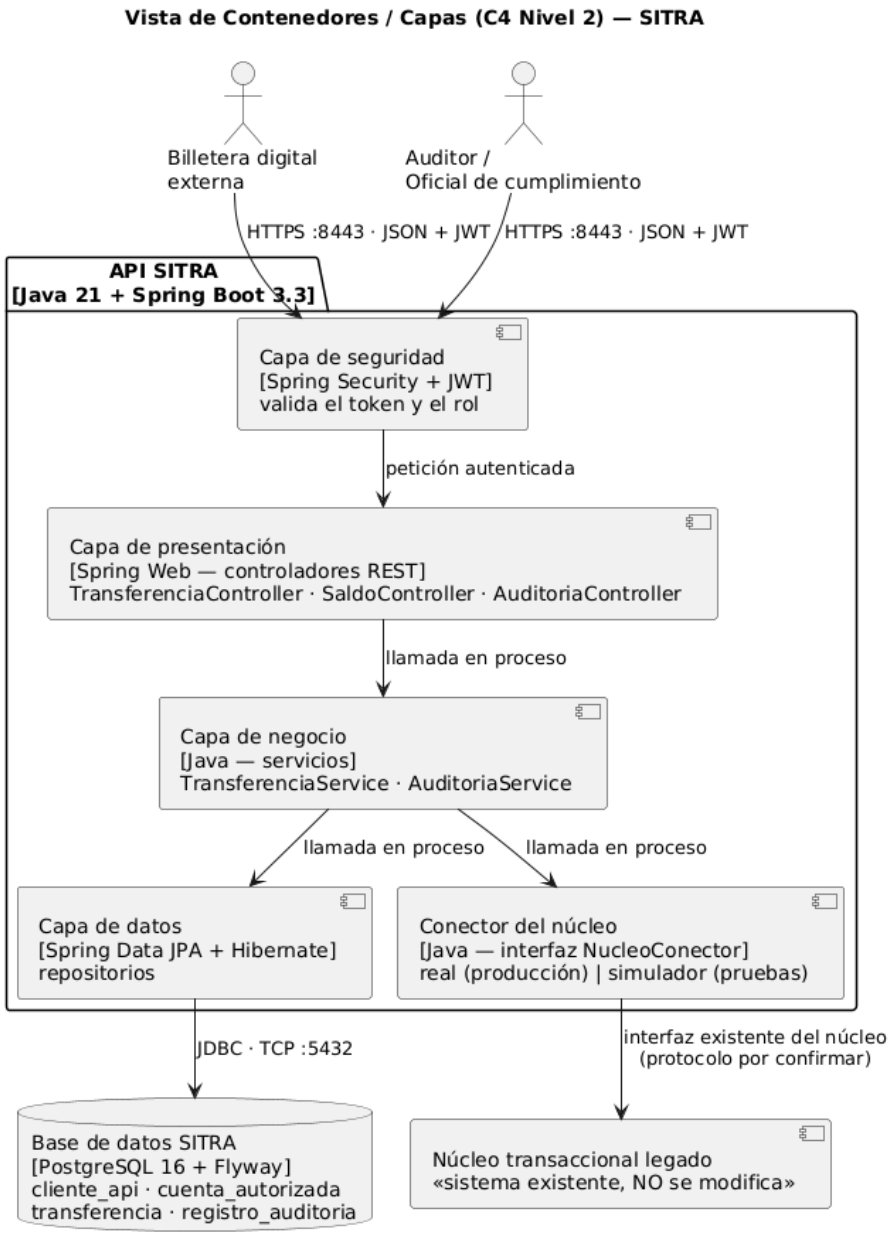


Figura 2. Vista de Contenedores: el contenedor API SITRA con sus capas y la base de datos.

Leyenda

- Marco: contenedor ejecutable (la aplicación). Cajas internas: capas. Cilindro: base de datos. Entre corchetes: tecnología concreta (la misma de la Sección 2.2).
- Etiquetas de las flechas: protocolo y puerto.

Lectura en 30 segundos

Hay solo dos contenedores: la API SITRA (Java 21 + Spring Boot 3.3, puerto 8443 por HTTPS) y la base de datos PostgreSQL 16 (puerto 5432, solo red interna). Dentro de la API, la petición baja por las capas: seguridad, presentación, negocio y, desde negocio, a datos o al conector del núcleo. El conector tiene dos versiones: la real, para producción, y un simulador, para pruebas; se cambia una por otra con configuración, sin tocar código.

3.3 Vista de Dominio — Diagrama de Clases UML

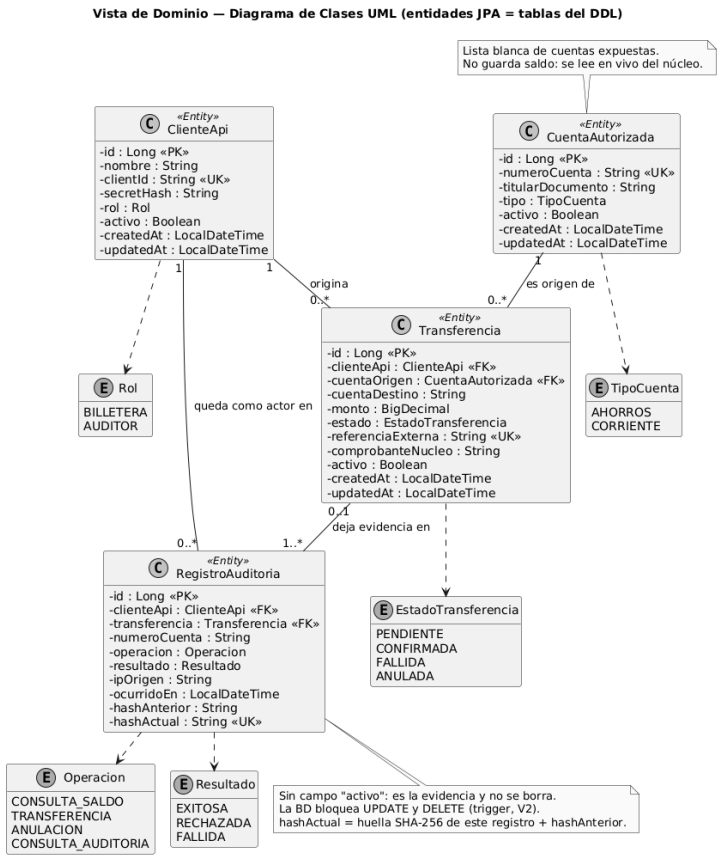


Figura 3. Diagrama de clases: entidades JPA, atributos tipados, llaves y multiplicidades.

Leyenda

- «PK» llave primaria, «FK» llave foránea, «UK» valor único. «Entity» = clase que JPA guarda como tabla. E = enumeración (valores permitidos).
- Números en las líneas: multiplicidad (1 = uno, 0..* = cero o muchos, 0..1 = cero o uno, 1..* = uno o muchos).

Lectura en 30 segundos

Cuatro entidades. Transferencia es la principal (HU-01): la origina un ClienteApi (la billetera) desde una CuentaAutorizada. Cada transferencia deja uno o más RegistroAuditoria, y cada registro guarda quién (clienteApi), qué (operacion), sobre qué cuenta, con qué resultado y cuándo. Dos decisiones para defender: CuentaAutorizada no guarda saldo, porque el saldo siempre se lee del núcleo; y RegistroAuditoria no tiene campo activo, porque es la evidencia y no se puede borrar de ninguna forma.

Correspondencia clase ↔ tabla

Clase (Java)	Tabla (PostgreSQL)	Borrado	Aparece en la secuencia (3.4)
ClienteApi	cliente_api	Lógico (activo)	Se identifica en FiltroJwt
CuentaAutorizada	cuenta_autorizada	Lógico (activo)	Es la cuenta origen de la transferencia (FK)
Transferencia	transferencia	Lógico (activo + ANULADA)	INSERT y UPDATE (pasos 9–10, 14–15, 22–23)
RegistroAuditoria	registro_auditoria	Ninguno (bloqueado por trigger)	INSERT bitácora (pasos 16–17, 24–25)

3.4 Vista Dinámica — Diagrama de Secuencia UML (HU-01)

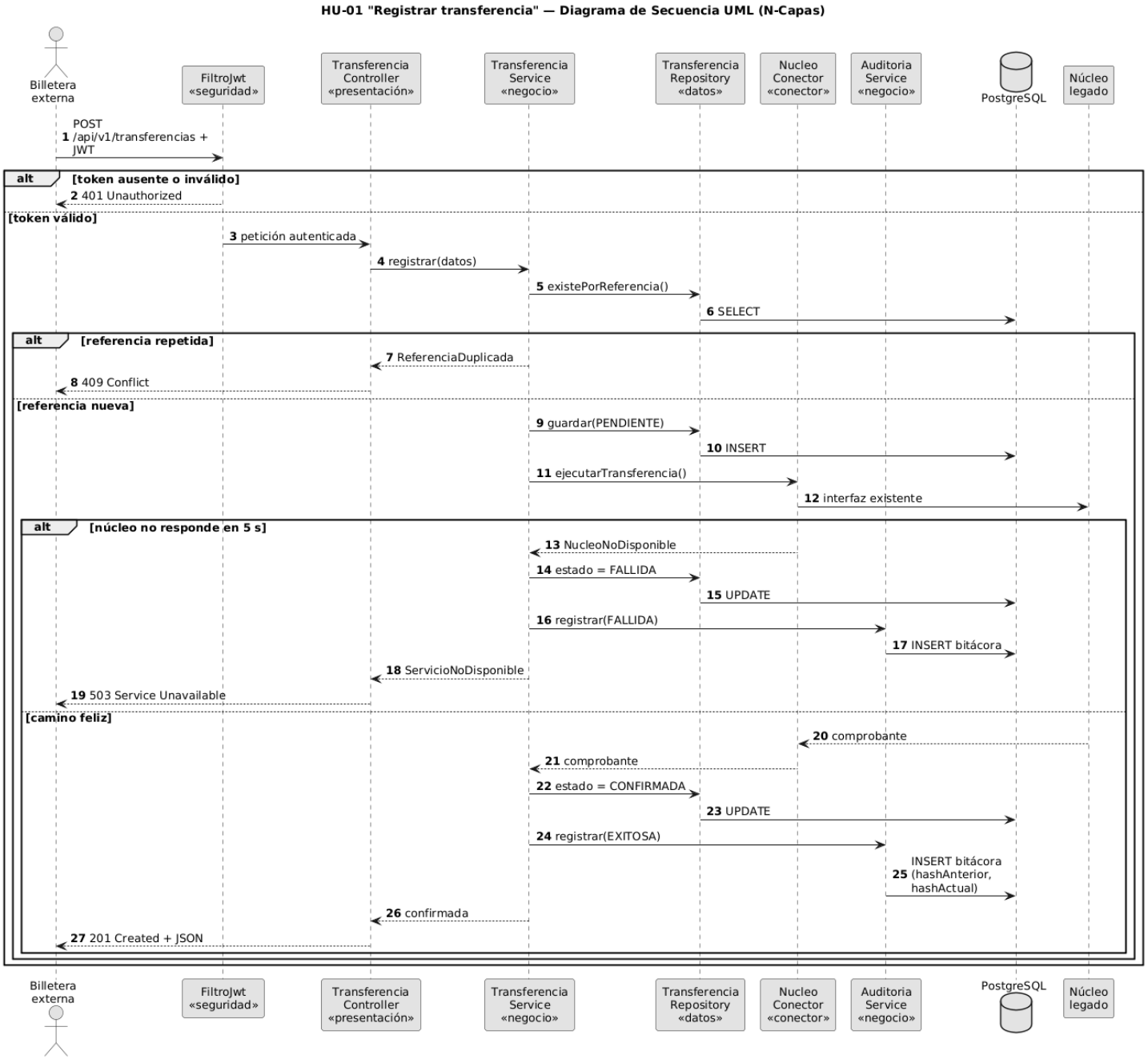


Figura 4. Secuencia de HU-01 "Registrar transferencia": camino feliz y tres flujos alternos (401, 409, 503).

Leyenda: flecha continua = llamada; flecha punteada = respuesta; recuadro alt = alternativas según una condición. Los pasos están numerados.

Lectura en 30 segundos: la petición atraviesa las capas en orden. Si no trae token válido, la seguridad responde 401 sin llegar al negocio. Si la referencia ya existe, el servicio responde 409 y no repite la operación. Si la referencia es nueva, se guarda como PENDIENTE y se llama al núcleo a través del conector: si el núcleo no responde en 5 s, la transferencia queda FALLIDA, se registra en la bitácora y se responde 503; si confirma, queda CONFIRMADA, se registra en la bitácora y se responde 201. En ambos casos queda evidencia, que es lo que pide el auditor.

3.5 Vista de Despliegue

Se entrega en la Clase 5 (24/10/2026), junto con el ADR y el arranque del corte vertical.

SECCIÓN 5.2: SCRIPT SQL DDL / MIGRACIONES

El esquema se divide en dos migraciones de Flyway, guardadas en db/migrations/ del repositorio. Se ejecutaron sobre PostgreSQL 16 y se verificaron con el script db/pruebas/verificar_restricciones.sql (resultado al final de esta sección).

db/migrations/V1__esquema_inicial.sql

```
-- ========================================================================
-- SITRA — Servicio de Integración y Trazabilidad (Caso 5, cooperativa)
-- V1: esquema inicial — PostgreSQL 16
-- Requisitos evaluables: PK en toda tabla, FK con REFERENCES, CHECK,
-- UNIQUE, borrado lógico (activo) en toda entidad de negocio e índices.
-- ========================================================================

-- Sistemas y personas autorizadas a usar SITRA (la billetera y el auditor).
CREATE TABLE cliente_api (
    id            BIGSERIAL     PRIMARY KEY,
    nombre        VARCHAR(120)  NOT NULL,
    client_id     VARCHAR(80)   NOT NULL UNIQUE,
    secret_hash   VARCHAR(255)  NOT NULL,                -- BCrypt, nunca el secreto en claro
    rol           VARCHAR(20)   NOT NULL DEFAULT 'BILLETERA'
                  CHECK (rol IN ('BILLETERA', 'AUDITOR')),
    activo        BOOLEAN       NOT NULL DEFAULT TRUE,    -- borrado lógico
    created_at    TIMESTAMP     NOT NULL DEFAULT NOW(),
    updated_at    TIMESTAMP     NOT NULL DEFAULT NOW()
);

-- Lista blanca: únicas cuentas que se pueden exponer a la billetera (RF-07).
-- NO guarda saldo: el saldo siempre se lee en vivo del núcleo (RF-01).
CREATE TABLE cuenta_autorizada (
    id                 BIGSERIAL    PRIMARY KEY,
    numero_cuenta      VARCHAR(30)  NOT NULL UNIQUE,
    titular_documento  VARCHAR(30)  NOT NULL,
    tipo               VARCHAR(20)  NOT NULL
                       CHECK (tipo IN ('AHORROS', 'CORRIENTE')),
    activo             BOOLEAN      NOT NULL DEFAULT TRUE, -- borrado lógico
    created_at         TIMESTAMP    NOT NULL DEFAULT NOW(),
    updated_at         TIMESTAMP    NOT NULL DEFAULT NOW()
);

-- Entidad principal de la HU-01 y del corte vertical.
CREATE TABLE transferencia (
    id                  BIGSERIAL      PRIMARY KEY,
    cliente_api_id      BIGINT         NOT NULL REFERENCES cliente_api(id),
    cuenta_origen_id    BIGINT         NOT NULL REFERENCES cuenta_autorizada(id),
    cuenta_destino      VARCHAR(30)    NOT NULL,
    monto               NUMERIC(14,2)  NOT NULL CHECK (monto > 0),
    estado              VARCHAR(20)    NOT NULL DEFAULT 'PENDIENTE'
                        CHECK (estado IN ('PENDIENTE', 'CONFIRMADA', 'FALLIDA', 'ANULADA')),
    referencia_externa  VARCHAR(80)    NOT NULL UNIQUE,   -- evita duplicados (RF-08, 409)
    comprobante_nucleo  VARCHAR(80),                      -- lo devuelve el núcleo al confirmar
    activo              BOOLEAN        NOT NULL DEFAULT TRUE, -- borrado lógico (RF-10)
    created_at          TIMESTAMP      NOT NULL DEFAULT NOW(),
    updated_at          TIMESTAMP      NOT NULL DEFAULT NOW()
);

-- Bitácora de auditoría (RF-03, RF-05, AC-01 Seguridad).
-- No tiene campo "activo": es la evidencia; no se borra ni lógica ni físicamente.
CREATE TABLE registro_auditoria (
    id                BIGSERIAL    PRIMARY KEY,
    cliente_api_id    BIGINT       NOT NULL REFERENCES cliente_api(id),   -- quién
    transferencia_id  BIGINT       REFERENCES transferencia(id),          -- vacío en consultas de saldo
    numero_cuenta     VARCHAR(30)  NOT NULL,                              -- sobre qué cuenta
    operacion         VARCHAR(30)  NOT NULL                               -- qué
                      CHECK (operacion IN ('CONSULTA_SALDO', 'TRANSFERENCIA', 'ANULACION',
'CONSULTA_AUDITORIA')),
    resultado         VARCHAR(20)  NOT NULL                               -- con qué resultado
                      CHECK (resultado IN ('EXITOSA', 'RECHAZADA', 'FALLIDA')),
    ip_origen         VARCHAR(45)  NOT NULL,
    ocurrido_en       TIMESTAMP    NOT NULL DEFAULT NOW(),                -- cuándo
    hash_anterior     CHAR(64)     NOT NULL,                              -- huella del registro previo
```

```
    hash_actual       CHAR(64)     NOT NULL UNIQUE                        -- huella de este registro
);

-- Índices justificados por las consultas de las HU
CREATE INDEX idx_transferencia_cuenta  ON transferencia(cuenta_origen_id);            -- historial por cuenta
CREATE INDEX idx_transferencia_activas ON transferencia(estado) WHERE activo = TRUE;  -- listado sin anuladas
CREATE INDEX idx_auditoria_cuenta_fecha ON registro_auditoria(numero_cuenta, ocurrido_en); -- HU-03: auditor
consulta por cuenta y rango de fechas (meta ≤ 5 s)
```

db/migrations/V2__bitacora_inmutable.sql

```
-- ======================================================================
-- SITRA — V2: la base de datos impide modificar o borrar la bitácora.
-- Así la regla no depende del código de la aplicación (AC-01: 0 registros modificables).
-- ======================================================================

CREATE OR REPLACE FUNCTION fn_bitacora_inmutable() RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION 'registro_auditoria no se puede modificar ni borrar (operación: %)', TG_OP;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_bitacora_inmutable
    BEFORE UPDATE OR DELETE ON registro_auditoria
    FOR EACH ROW EXECUTE FUNCTION fn_bitacora_inmutable();
```

Cumplimiento de los requisitos evaluables

Requisito de la plantilla	Dónde se cumple
PK en toda tabla	id BIGSERIAL PRIMARY KEY en las 4 tablas.
FK con REFERENCES	transferencia → cliente_api y cuenta_autorizada; registro_auditoria → cliente_api y transferencia.
Al menos una CHECK	rol, tipo de cuenta, monto > 0, estado, operacion y resultado.
Al menos una UNIQUE	client_id, numero_cuenta, referencia_externa (evita transferencias duplicadas → 409) y hash_actual.
Borrado lógico en toda entidad de negocio	Campo activo en cliente_api, cuenta_autorizada y transferencia. Anular una transferencia = activo FALSE + estado ANULADA.
Índices justificados	idx_transferencia_cuenta (historial por cuenta); idx_transferencia_activas (listado sin anuladas); idx_auditoria_cuenta_fecha (HU-03: el auditor consulta por cuenta y rango de fechas, meta ≤ 5 s).

Excepción defendible: registro_auditoria no tiene campo activo. No es una entidad de negocio sino la evidencia del sistema: si se pudiera marcar como inactiva, dejaría de probar algo ante el auditor. La migración V2 lo hace cumplir en la propia base de datos con un trigger que rechaza cualquier UPDATE o DELETE sobre esa tabla.

Verificación ejecutada (PostgreSQL 16)

```
$ psql -d sitra -f db/migrations/V1__esquema_inicial.sql
$ psql -d sitra -f db/migrations/V2__bitacora_inmutable.sql
$ psql -d sitra -f db/pruebas/verificar_restricciones.sql
NOTICE:  OK 1: UNIQUE rechaza referencia duplicada
NOTICE:  OK 2: CHECK rechaza monto negativo
NOTICE:  OK 3: FK rechaza cuenta inexistente
NOTICE:  OK 4: CHECK rechaza estado inválido
NOTICE:  OK 5: trigger impide modificar la bitácora
NOTICE:  OK 6: trigger impide borrar la bitácora
NOTICE:  OK 7: borrado lógico conserva la transferencia y su registro de auditoría
```

Secciones pendientes

Sección	Contenido	Clase
3.5 y 4	Vista de despliegue y ADR (decisiones de arquitectura).	Clase 5 — 24/10
5.1 y 5.3	Estructura del código y CRUD punta a punta de Transferencia.	Clases 5 y 6
5.4 y 6	Pruebas, análisis de trade-offs y riesgos.	Clase 7 — 14/11